

Metrics for classification and object detection AI models

When we talk about “classification models”, we usually mean a model that assigns one label to an image, such as “cat” or “dog”. In our project, however, YOLO does something different: it performs object detection. That means it predicts what the object is and where it is in the image, using bounding boxes. Because of that extra “where” part, evaluation is stricter than for pure classification, and the most important metrics (such as mAP) are built around both label correctness and box quality.

A good evaluation should therefore answer three questions at the same time. First, does the model detect the right objects at all? Second, does it confuse classes (for example bike vs motorbike)? Lastly, if it detects the correct object, how accurate is the predicted box compared to the ground-truth box?

The role of IoU in object detection

The key geometric concept behind detection metrics is **Intersection over Union (IoU)**. IoU measures how much the predicted box overlaps with the ground-truth box. If the overlap is perfect, IoU is 1. If the boxes do not overlap, IoU is 0. In evaluation we choose an IoU threshold to decide whether a detection counts as a correct match. For example, “IoU = 0.50” is relatively forgiving and accepts boxes that are roughly right, while “IoU = 0.75” or “IoU = 0.70” is stricter and requires the box to be much tighter around the object.

This is important: a model can look “good” at low IoU thresholds while still producing sloppy boxes, especially for small objects. Higher IoU thresholds punish that behaviour.

Precision, recall and why confidence thresholds matter

YOLO outputs many candidate boxes with a confidence score. By setting a confidence threshold, we decide which predictions we keep. This threshold directly affects two core metrics.

- **Precision** answers: “When the model predicts an object, how often is it correct?” High precision means few false positives, so the model is not hallucinating objects that are not there.
- **Recall** answers: “Of all real objects in the image, how many does the model find?” High recall means few false negatives, so the model misses fewer objects.

There is an unavoidable trade-off. If you raise the confidence threshold, the model becomes more conservative. Precision usually improves, but recall often drops because you throw away uncertain detections. If you lower the threshold, recall can improve, but precision tends to drop because you accept more weak predictions.

This is why it is misleading to report precision or recall at only one arbitrary confidence threshold without context. A single threshold reflects one operating point, not the overall behaviour of the model.

F1-score as a balance between precision and recall

To summarise the precision–recall trade-off in one number, we often use the **F1-score**, which is the harmonic mean of precision and recall. The harmonic mean is intentionally strict: if either precision or recall is low, F1 will also be low. In practice, F1 is useful when you want a balanced model that both finds objects and avoids false alarms.

However, it is still threshold-dependent. A model can have a strong F1 at one confidence threshold and a weaker F1 at another. For that reason, detection papers and benchmarks usually rely more heavily on metrics that integrate performance over many thresholds.

Average Precision and the Precision–Recall curve

Instead of fixing one confidence threshold, we can evaluate the model across all thresholds by creating a **Precision–Recall (PR) curve**. This curve shows how precision changes as recall increases. A better model maintains high precision even when recall becomes high.

Average Precision (AP) summarises the PR curve into one number by computing the area under that curve. AP is computed per class. That means you can get one AP value for “vehicle”, one for “pedestrian”, and so on. This is very useful because it shows where the model is strong and where it struggles, rather than hiding everything behind one global score.

A critical detail is that AP is only meaningful when the evaluation protocol is consistent. If you change dataset splits, IoU rules, or filtering settings, AP values cannot be compared fairly anymore.

Because we usually have multiple classes, we take the AP of each class and average them. This gives **mAP**, or mean Average Precision. mAP is the most common “headline number” for object detection because it captures both classification correctness and localisation quality, and it does so across confidence thresholds.

Still, mAP is not a single universal number. It always depends on the IoU threshold(s) used.

mAP50, mAP75, and mAP50–95

When you see mAP50, it means mAP computed with IoU threshold fixed at 0.50. This score is easier to improve and mostly tests whether the model finds the object in approximately the right place.

mAP75 uses IoU threshold 0.75, which is stricter and rewards tighter boxes.

The most widely used modern benchmark metric is **mAP50–95**, sometimes written as mAP@[0.50:0.95] . This averages mAP over multiple IoU thresholds from 0.50 to 0.95 in steps of 0.05. It is the most honest summary because it forces the model to perform well not only on “roughly correct” boxes but also on very accurate boxes.

This also explains why mAP50 is often much higher than mAP50–95: it is simply a less strict requirement.

If you see a setting like “IoU = 0.70”, that is not automatically a standard mAP metric by itself, but it refers to the matching rule used during evaluation. You can compute mAP at that threshold too, but it should be clearly stated as mAP@0.70 to avoid confusion.

Confusion matrix and per-class behaviour

A confusion matrix is especially useful when classes are visually similar, such as “bike” and “motorbike”. It shows which classes are commonly confused. For detection, confusion can come from two sources. The model may predict the wrong class for the right object, or it may fail to detect the object and instead detect something else nearby. Looking at per-class metrics together with confusion patterns is often more informative than a single global mAP.

A model can also look strong in overall mAP while still failing on a rare but important class. This often happens in imbalanced datasets, where frequent classes dominate the average.

What these metrics do not tell you

Even a solid set of metrics can hide important weaknesses. mAP and F1 do not fully capture how the model behaves in edge cases like motion blur, night scenes, unusual camera angles, or heavy occlusion. They also do not directly measure temporal stability in video, where flickering detections can be a real problem even if per-frame mAP is good.

Because of that, the most honest evaluation combines quantitative metrics with qualitative inspection of predictions, especially on failure cases that matter for the intended use.

How to report results in a fair way

A clean workflow is to train on the training split, use validation to choose settings and select the best checkpoint and then report final numbers on a separate test split. This is important because validation results influence decisions like early stopping and hyperparameter choices. If we repeatedly optimise using the validation set, validation becomes less “neutral”, and the test set becomes the only honest estimate of final generalisation.